

Module 7 Lab:

Wireshark Analysis: Network Protocols and Layered Vulnerabilities

JR Clark (002035589)

NEU DAMG7350 (Spring 2026)

Introduction

This lab used Wireshark to examine how different network protocols expose information at various layers of the OSI and TCP/IP models. By capturing live traffic, applying display filters, and analyzing packets across multiple layers, I was able to observe how metadata, plaintext protocols, and handshake negotiations reveal operational details that attackers could exploit. This aligns with the module's emphasis on understanding networks as layered ecosystems where vulnerabilities emerge at different points in the stack (Sawant, 2026).

1. OSI and TCP/IP Layers of the Sample Capture

The sample capture provided in the lab instructions—specifically the insecure HTTP GET request to `example.com`—corresponds to the Application Layer in both the OSI and TCP/IP models. HTTP operates entirely at the top of the stack, carrying user-facing data such as request methods, headers, and content. In the sample capture, Wireshark displays the full plaintext request, including the Host header, User-Agent string, and Accept fields, which are all characteristic of OSI Layer 7 behavior.

In the TCP/IP model, this same traffic maps to the Application Layer, which consolidates OSI Layers 5–7 into a single functional tier. As the Coursera module notes, the TCP/IP Application layer “combines OSI’s top three layers: application, presentation, and session,” placing HTTP squarely within this layer (Sawant, 2026). Because the sample packet contains no encryption negotiation or transport-layer control information beyond the TCP header, its highest-level protocol—and therefore its classification—is unambiguously Application-layer traffic.

This sample establishes a baseline for the lab: a plaintext, high-level protocol that exposes complete request details. The following sections extend this analysis to three packets captured from my own live traffic.

2. Packet Analysis #1 — TLS ClientHello (OSI Layer 6 / TCP-IP Application Layer)

The first packet I selected was a TLSv1.3 ClientHello sent from my VM to trends.google.com. Although TLS ultimately encrypts application data, the handshake phase exposes significant metadata (see Step 13 in Appendix). In the Packet Details pane, I observed the full list of cipher suites offered by the client:

- Supported TLS versions (1.2 and 1.3)
- The Server Name Indication (SNI) extension revealing the hostname
- Key share groups and signature algorithms

This packet corresponds to the Presentation Layer (Layer 6) in the OSI model because it negotiates encryption parameters and prepares data for secure transmission. In the TCP/IP model, TLS is part of the Application Layer, which includes all protocols above the transport layer.

The visibility of SNI and cipher suite preferences illustrates a key point from the module: even secure protocols leak metadata during negotiation, enabling fingerprinting and traffic analysis (Sawant, 2026). While the encrypted session protects confidentiality, the handshake reveals enough information for an observer to infer the destination service and client capabilities.

3. Packet Analysis #2 — DNS Query (OSI Layer 7 / TCP-IP Application Layer)

The second packet was a DNS query sent from 10.0.2.15 to the local DNS resolver at 192.168.1.1. The packet contained a standard query header, transaction ID, and the queried domain name (see Step 14 in Appendix). DNS operates at the Application Layer (Layer 7) in the OSI model and the Application Layer in TCP/IP.

Because DNS is transmitted in plaintext, it exposes: the exact domain being queried, the client's internal IP address, the DNS server being used, timing and frequency of lookups.

This aligns with the module's discussion of how application-layer protocols can leak sensitive information even when higher-level traffic is encrypted. DNS metadata can reveal browsing behavior, device activity patterns, and potential targets for phishing or spoofing attacks (Sawant, 2026).

4. Packet Analysis #3 — HTTP POST with OCSP Request (OSI Layer 7 / TCP-IP Application Layer)

The third packet was an HTTP/1.1 POST request to o.pki.goog, carrying an OCSP request used to validate certificate status (see Step 15 in Appendix). Because this OCSP traffic was sent over port 80, the full HTTP headers—including Host, User-Agent, Content-Type, and Content-Length—were visible in plaintext.

This packet belongs to the Application Layer (Layer 7) in both OSI and TCP/IP models. Although OCSP is part of a security mechanism, many implementations still rely on HTTP rather than HTTPS, which exposes metadata about certificate validation behavior. This reflects the module's emphasis on how even security-related protocols can introduce vulnerabilities when implemented without encryption or proper configuration (Sawant, 2026).

5. Cross-Layer Comparison

Analyzing these three packets together highlights how different layers expose different types of information. The TLS ClientHello (Layer 6) revealed encryption negotiation details but not application content. The DNS query (Layer 7) exposed domain lookups in plaintext,

revealing browsing behavior. The OCSP HTTP POST (Layer 7) exposed full HTTP headers and metadata about certificate validation.

These observations reinforce the module's framing of layered security: vulnerabilities at one layer can undermine protections at another. Even when application data is encrypted, metadata leakage at the presentation or application layers can reveal sensitive information about user behavior, device configuration, and communication patterns.

Conclusion

This lab demonstrated how Wireshark can be used to analyze network protocols across multiple layers and identify the information each protocol exposes. By examining TLS, DNS, and HTTP/OCSP packets, I observed how metadata leakage, plaintext protocols, and handshake negotiation all contribute to the broader threat landscape described in Module 7. Understanding these patterns is essential for designing layered defenses that address vulnerabilities at each stage of packet processing.

References

Sawant, A. (2026). Network Protocols and Threats [Online course]. Coursera & Northeastern University.

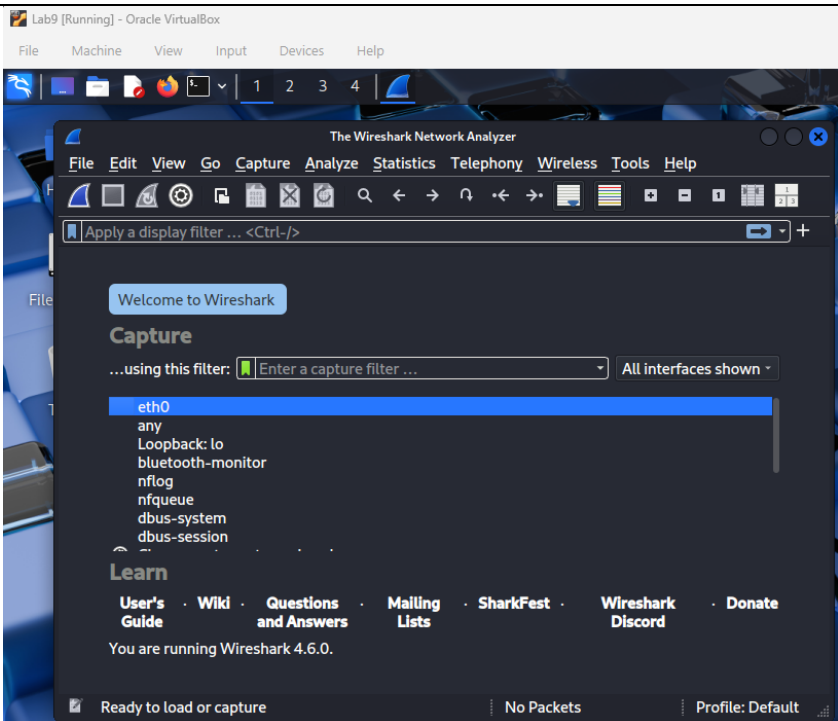
Methodology and Use of AI Tools

I completed all technical steps of this lab directly in the Kali Linux virtual machine, including packet capture, filter creation, profile configuration, and detailed analysis of TLS, DNS, and HTTP/OCSP traffic. All screenshots and observations were generated from my own live capture sessions. I used Microsoft Copilot as a writing and organizational aid to help structure the report, refine transitions between sections, and ensure alignment with the assignment rubric. Copilot did not perform any technical steps in Wireshark or interpret packet data on my behalf. All analysis, interpretations, and conclusions reflect my own understanding of the material and the concepts covered in Module 7.

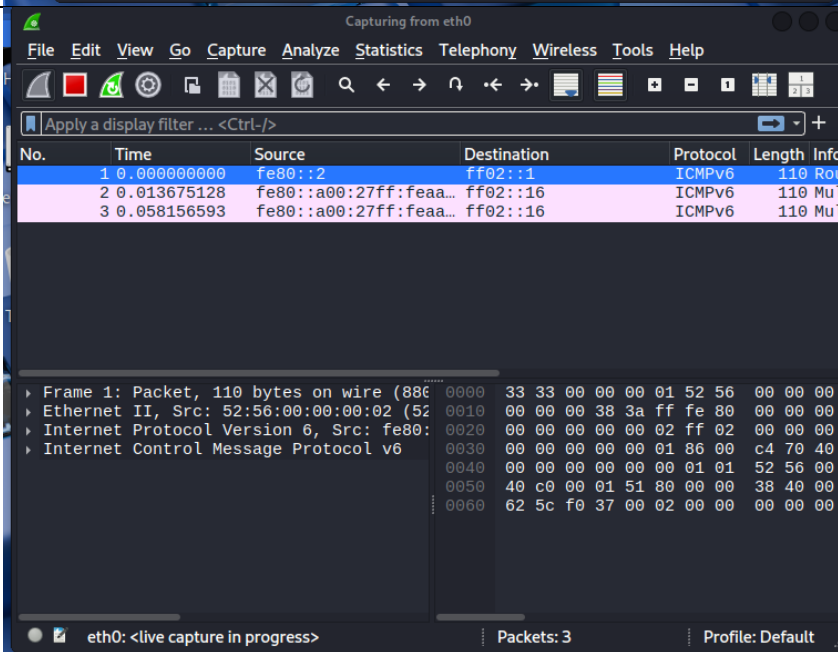
APPENDIX: Documented Process

The following table documents the exact steps performed in Wireshark, with each left-column description corresponding to the screenshot or output shown in the right column.

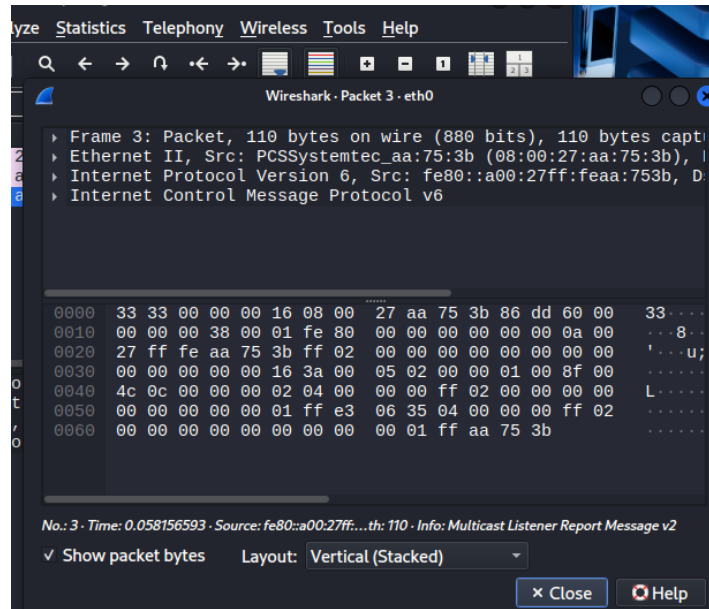
Step 1: Clone Kali VM for Lab 9, login, open Wireshark

The image shows the Wireshark Network Analyzer interface. At the top, there's a menu bar with options like File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. Below the menu is a toolbar with various icons. A search bar is present with the text 'Apply a display filter ... <Ctrl-/>'. The main area is titled 'Welcome to Wireshark' and 'Capture'. It shows a list of network interfaces: eth0, any, Loopback: lo, bluetooth-monitor, nflog, nfqueue, dbus-system, and dbus-session. The 'eth0' interface is highlighted. Below the list, there's a 'Learn' section with links to User's Guide, Wiki, Questions and Answers, Mailing Lists, SharkFest, Wireshark Discord, and Donate. At the bottom, the status bar shows 'Ready to load or capture', 'No Packets', and 'Profile: Default'.

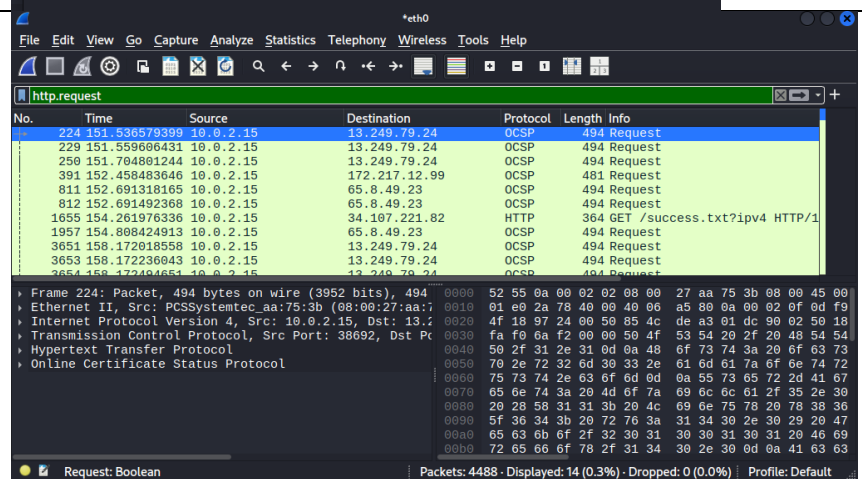
Step 2: Start capturing from eth0

The image shows the Wireshark Network Analyzer interface with active packet capture. The status bar at the top indicates 'Capturing from eth0'. The menu bar and toolbar are the same as in the previous image. The search bar still shows 'Apply a display filter ... <Ctrl-/>'. The main area now displays a list of captured packets. The first packet is highlighted, showing its details in the packet details pane. The packet list table has columns: No., Time, Source, Destination, Protocol, Length, and Info. The first packet is a Frame 1, 110 bytes on wire, from Source fe80::2 to Destination ff02::1, using ICMPv6. The packet details pane shows the structure of the first packet: Frame 1: Packet, 110 bytes on wire (880 bytes captured) on interface eth0, Ethernet II, Src: 52:56:00:00:00:02 (52:56:00:00:00:02), Dst: 01:00:5e:00:00:01 (01:00:5e:00:00:01), Internet Protocol Version 6, Src: fe80::2, Dst: ff02::1, and Internet Control Message Protocol v6. The status bar at the bottom shows 'eth0: <live capture in progress>', 'Packets: 3', and 'Profile: Default'.

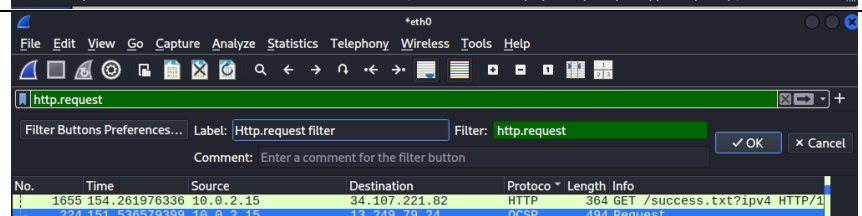
Step 3: View packet details



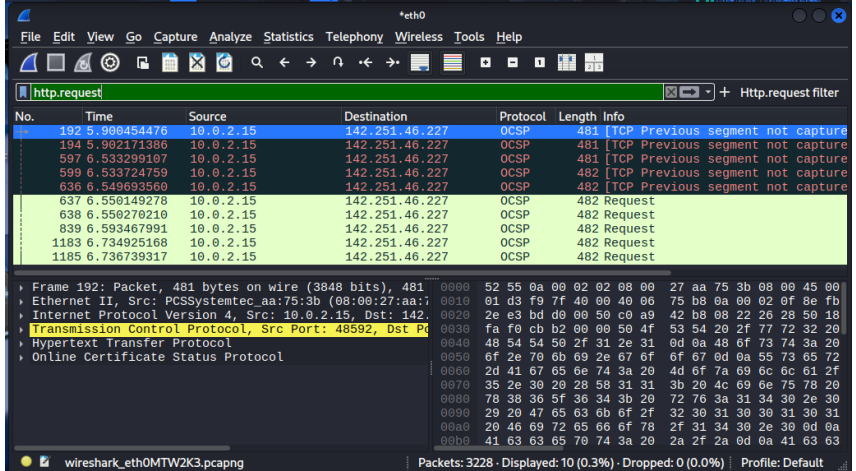
Step 4: Create a filter for http.request after visiting example.com



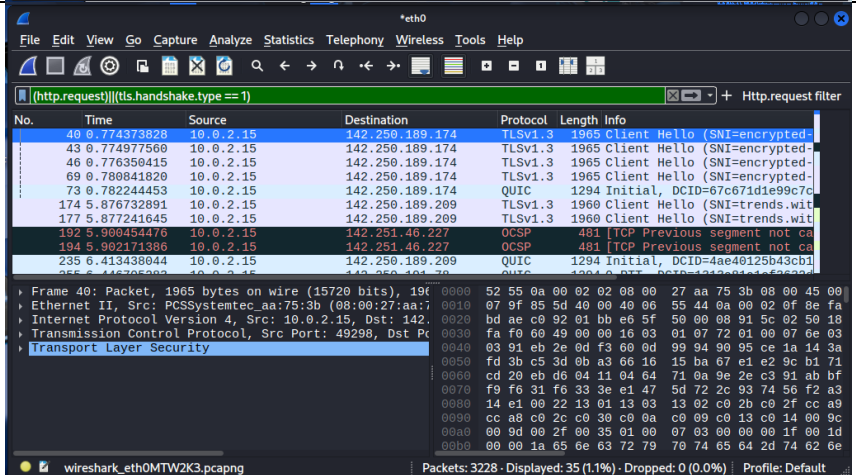
Step 5a: saving http.request filter and browse more websites



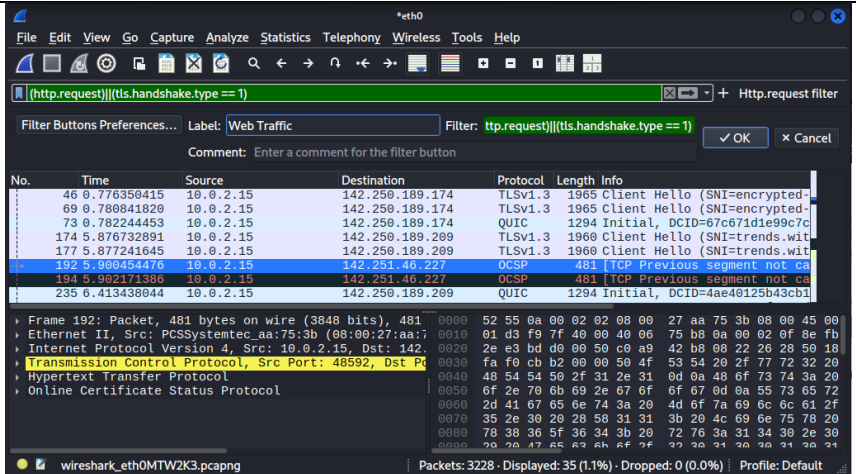
Step 5b: browsing more websites



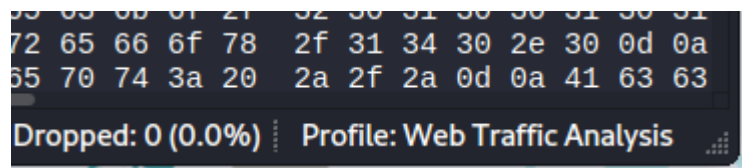
Step 6:
(http.request)||((tls.handshake.type == 1))



Step 7: save Web Traffic filter



Step 8: Create new profile
“Web Traffic Analysis”



Step 9: Configure additional columns and update time format

Wireshark - Preferences

Displayed	Title	Type	Custom Expression	Field
☒	No.	Number		
☒	Time	Time (format as specified)		
☒	Source	Source address		
☒	Src Port	Source port		
☒	Dest Port	Destination port		
☒	Destination	Destination address		
☒	Protocol	Protocol		
☒	Host name	Custom	http.host tls.handshake.extensions_server_name	0
☒	Length	Packet length (bytes)		
☒	Info	Information		

Time

1183	2026-02-19	19:49:30.338554705Z
1162	2026-02-19	19:49:30.317180818Z
1156	2026-02-19	19:49:30.315007498Z
1142	2026-02-19	19:49:30.307446496Z
1028	2026-02-19	19:49:30.248578113Z
839	2026-02-19	19:49:30.197097528Z
638	2026-02-19	19:49:30.153899747Z
637	2026-02-19	19:49:30.153778815Z

Explore Vulnerable Packets

Step 10: Download sample file and open in Wireshark

Wireshark - Open Capture File

Look in: /home/jenn/Downloads

Computer	Name	Size	Type	Date Modified
jenn	nb6-http.pcap	8.60 KIB	Pac...AP)	2/19/26 12:12 PM

Step 11: Follow the http stream

Wireshark - Follow HTTP Stream (tcp.stream eq 0) - nb6-http.pcap

```
GET /nb6dsl_Vers%203.3.4_ter/nb6-3.3.4_ter.sha256sum HTTP/1.1
User-Agent: curl/7.33.0
Host: ncdn.nb6dsl.neufbox.neuf.fr
Accept: */*

HTTP/1.1 200 OK
Server: Apache
Last-Modified: Thu, 01 Aug 2013 18:16:36 GMT
Cache-Control: max-age=86400
Expires: Fri, 03 Jan 2014 02:45:16 GMT
X-Varnish: 1740521562
Content-Length: 497
Accept-Ranges: bytes
Date: Thu, 02 Jan 2014 08:37:49 GMT
X-Varnish: 3638095267 3619940513
Age: 21153
Connection: keep-alive
Via: 1.1 varnish, 1.1 cbv4-ncdn-cache06, 1.1 cbv4-ncdn-cache00

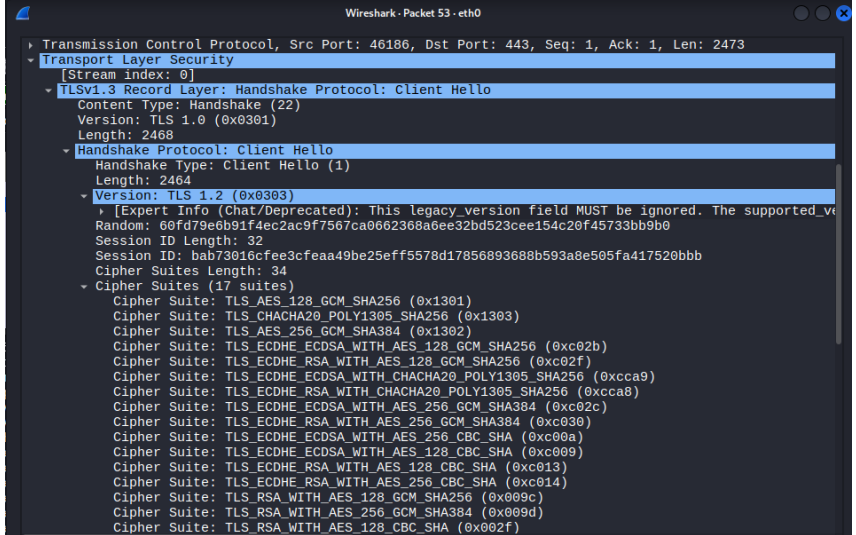
#
# UUID: 27f4b1e0-6628-49cb-b59b-dc2234dc0e5d
#
# NB6 3.3.4 sha256sum
#

1 client pkt, 1 server pkt, 1 turn.
```

Step 12: Choose a different OSI layer and TCP/IP layer and capture packets – live capture on own network selected:

Title	Protocol	Type	Protocol	Fields	Occurrence	Display as	Strings	OK	Canc
Time	Source	Src Port	Dest Port	Destination	Protocol	Host name	Length	Info	
50	2026-02-19 2. 10.0.2.15	46186	443	142.250.189.196	TCP		74	46186 → 443 [SYN] Seq=...	
51	2026-02-19 2. 142.250.189.196	443	46186	10.0.2.15	TCP		60	443 → 46186 [SYN, ACK] Seq=...	
52	2026-02-19 2. 10.0.2.15	46186	443	142.250.189.196	TCP		54	46186 → 443 [ACK] Seq=...	
53	2026-02-19 2. 10.0.2.15	46186	443	142.250.189.196	TLSv1.3	trends.g...	2527	client Hello (SN=17ren...	
54	2026-02-19 2. 142.250.189.196	443	46186	10.0.2.15	TCP		60	443 → 46186 [ACK] Seq=...	
55	2026-02-19 2. 142.250.189.196	443	46186	10.0.2.15	TCP		60	443 → 46186 [ACK] Seq=...	
56	2026-02-19 2. 10.0.2.15	46186	443	142.250.189.196	TLSv1.3		152	Change Cipher Spec, Ap...	
57	2026-02-19 2. 142.250.189.196	443	46186	10.0.2.15	TCP		60	443 → 46186 [ACK] Seq=...	
65	2026-02-19 2. 142.250.189.196	443	46186	10.0.2.15	TLSv1.3		2065	Server Hello, Change C...	
66	2026-02-19 2. 10.0.2.15	46186	443	142.250.189.196	TCP		54	46186 → 443 [ACK] Seq=...	
67	2026-02-19 2. 10.0.2.15	46186	443	142.250.189.196	TLSv1.3		138	Application Data, AppL...	
68	2026-02-19 2. 142.250.189.196	443	46186	10.0.2.15	TCP		60	443 → 46186 [ACK] Seq=...	
69	2026-02-19 2. 10.0.2.15	46186	443	142.250.189.196	TLSv1.3		85	Application Data	
70	2026-02-19 2. 142.250.189.196	443	46186	10.0.2.15	TCP		60	443 → 46186 [ACK] Seq=...	
39326	2026-02-19 2. 10.0.2.15	46186	443	142.250.189.196	TLSv1.3		93	Application Data	

Step 13: select 3 packets for analysis: Packet #1
TLS/Client Hello client



```
Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (0xc009)
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013)
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014)
Cipher Suite: TLS_RSA_WITH_AES_128_GCM_SHA256 (0x009c)
Cipher Suite: TLS_RSA_WITH_AES_256_GCM_SHA384 (0x009d)
Cipher Suite: TLS_RSA_WITH_AES_128_CBC_SHA (0x002f)
Cipher Suite: TLS_RSA_WITH_AES_256_CBC_SHA (0x0035)
Compression Methods Length: 1
Compression Methods (1 method)
Extensions Length: 2357
Extension: server_name (len=22) name=trends.google.com
Type: server_name (0)
Length: 22
  Server Name Indication extension
Extension: extended_master_secret (len=0)
Extension: renegotiation_info (len=1)
Extension: supported_groups (len=16)
Extension: ec_point_formats (len=2)
Extension: application_layer_protocol_negotiation (len=14)
Extension: status_request (len=5)
Extension: delegated_credentials (len=10)
Extension: signed_certificate_timestamp (len=0)
Extension: key_share (len=1327) X25519MLKEM768, x25519, secp256r1
Type: key_share (51)
Length: 1327
  Key Share extension
Extension: early_data (len=0)
Extension: supported_versions (len=5) TLS 1.3, TLS 1.2
Extension: signature_algorithms (len=24)
Extension: psk_key_exchange_modes (len=2)
Extension: record_size_limit (len=2)
Extension: compress_certificate (len=7)
Extension: encrypted_client_hello (len=569)
```

Step 14: select 3 packets for analysis: DNS Packet
#2 (80)

<

```
Wireshark - Packet 80 - eth0

▼ Frame 80: Packet, 75 bytes on wire (600 bits), 75 bytes captured (600 bits) on interface eth0, id 0
  Section number: 1
  ▸ Interface id: 0 (eth0)
    Encapsulation type: Ethernet (1)
    Arrival Time: Feb 19, 2026 12:24:04.392610524 PST
    UTC Arrival Time: Feb 19, 2026 20:24:04.392610524 UTC
    Epoch Arrival Time: 1771532644.392610524
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 69.308 microseconds]
    [Time delta from previous displayed frame: 115.812 microseconds]
    [Time since reference or first frame: 2.006346800 seconds]
    Frame Number: 80
    Frame Length: 75 bytes (600 bits)
    Capture Length: 75 bytes (600 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:udp:dns]
    Character encoding: ASCII (0)
    [Coloring Rule Name: UDP]
    [Coloring Rule String: udp]
  ▼ Ethernet II, Src: PCSSystemtec_aa:75:3b (08:00:27:aa:75:3b), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
    ▸ Destination: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
      .... 1. .... = LG bit: Locally administered address (this is NOT the factory default)
      .... 0. .... = IG bit: Individual address (unicast)
    ▸ Source: PCSSystemtec_aa:75:3b (08:00:27:aa:75:3b)
      .... 0. .... = LG bit: Globally unique address (factory default)
      .... 0. .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  ▼ Internet Protocol Version 4, Src: 10.0.2.15, Dst: 192.168.1.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
```

```
Wireshark - Packet 80 - eth0

    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 61
    Identification: 0xe270 (57968)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x8a87 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.2.15
    Destination Address: 192.168.1.1
    [Stream index: 0]
  ▼ User Datagram Protocol, Src Port: 34436, Dst Port: 53
    Source Port: 34436
    Destination Port: 53
    Length: 41
    Checksum: 0xcdcf2 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 10]
    [Stream Packet Number: 2]
  ▸ [Timestamps]
    UDP payload (33 bytes)
  ▼ Domain Name System (query)
    Transaction ID: 0x0f91
  ▸ Flags: 0x0100 Standard query
    Questions: 1
    Answer RRs: 0
    Authority RRs: 0
    Additional RRs: 0
  ▸ Queries
    [Response In: 84]
```

OCSP									
Title: Protocol	Time	Type: Protocol	Fields: Enter a cu...	Occurrence:	Display as: Strings	✓ OK	✕ Cancel		
No.		Source	Src Port	Dest Port	Destination	Protoco	Host name	Length	Info
267	2026-02-19 2...	10.0.2.15	36176	80	192.251.46.227	OCSP	o.pki.go...	1157	Request
321	2026-02-19 2...	142.251.46.227	80	36170	10.0.2.15	OCSP		1157	Response

Step 15: select 3 packets for analysis: OCSP (TCP/HTTP) Packet #3 (267)

```
Wireshark - Packet 267 - eth0

Frame 267: Packet, 482 bytes on wire (3856 bits), 482 bytes captured (3856 bits) on interface eth0,
  Section number: 1
  Interface id: 0 (eth0)
  Encapsulation type: Ethernet (1)
  Arrival Time: Feb 19, 2026 12:24:04.445225754 PST
  UTC Arrival Time: Feb 19, 2026 20:24:04.445225754 UTC
  Epoch Arrival Time: 1771532644.445225754
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 471.534 microseconds]
  [Time since reference or first frame: 2.058962030 seconds]
  Frame Number: 267
  Frame Length: 482 bytes (3856 bits)
  Capture Length: 482 bytes (3856 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:tcp:http:ocsp]
  Character encoding: ASCII (0)
  [Coloring Rule Name: HTTP]
  [Coloring Rule String: http || tcp.port == 80 || http2]
  Ethernet II, Src: PCSSystemtec_aa:75:3b (08:00:27:aa:75:3b), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
  Destination: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
    ....01.... = LG bit: Locally administered address (this is NOT the factory default)
    ....0.... = IG bit: Individual address (unicast)
  Source: PCSSystemtec_aa:75:3b (08:00:27:aa:75:3b)
    ....0.... = LG bit: Globally unique address (factory default)
    ....0.... = IG bit: Individual address (unicast)
  Type: IPv4 (0x0800)
  [Stream index: 0]
  Internet Protocol Version 4, Src: 10.0.2.15, Dst: 142.251.46.227
  0100 .... = Version: 4
  ....0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 468

0000 52 55 0a 00 02 02 08 00 27 aa 75 3b 08 00 45 00  RU ..... 'u';-E-

No: 267, Time: 2026-02-19 20:24:04.445225754Z, Source: 10.0.2.15, Src Port: 36170, Dst Port: 255146227, Protocol: OCSP, Host name: onkiogoo, Length: 482, Info: Request
```

```
Wireshark - Packet 267 - eth0

Total Length: 468
Identification: 0x53c2 (21442)
  010. .... = Flags: 0x2, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: TCP (6)
  Header Checksum: 0x1b75 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 10.0.2.15
  Destination Address: 142.251.46.227
  [Stream index: 13]
  Transmission Control Protocol, Src Port: 36170, Dst Port: 80, Seq: 1, Ack: 1, Len: 428
  Source Port: 36170
  Destination Port: 80
  [Stream index: 10]
  [Stream Packet Number: 4]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 428]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 3825955466
  [Next Sequence Number: 429 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 409728002
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x018 (PSH, ACK)
  Window: 64240
  [Calculated window size: 64240]
  [Window size scaling factor: -2 (no window scaling used)]
  Checksum: 0xcbb3 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]
  [SEQ/ACK analysis]
```

Wireshark - Packet 267 - eth0

0101 = Header Length: 20 bytes (5)

Flags: 0x018 (PSH, ACK)

Window: 64240

[Calculated window size: 64240]

[Window size scaling factor: -2 (no window scaling used)]

Checksum: 0xcbb3 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

[Timestamps]

[SEQ/ACK analysis]

[Client Contiguous Streams: 1]

[Server Contiguous Streams: 1]

TCP payload (428 bytes)

Hypertext Transfer Protocol

POST /wr2 HTTP/1.1\r\n

Host: o.pki.goog\r\n

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0\r\n

Accept: */*\r\n

Accept-Language: en-US,en;q=0.5\r\n

Accept-Encoding: gzip, deflate\r\n

Content-Type: application/ocsp-request\r\n

Content-Length: 84\r\n

Connection: keep-alive\r\n

Priority: u=2\r\n

Pragma: no-cache\r\n

Cache-Control: no-cache\r\n

\r\n

[Response in frame: 321]

[Full request URI: http://o.pki.goog/wr2]

File Data: 84 bytes

Online Certificate Status Protocol

tbsRequest